Gensyn Automated Settlement and Permit

Security Assessment

July 14, 2026

Prepared for:
Harry Grieve
Gensyn

Prepared by: **Guillermo Larregay**

Table of Contents

Project Summary

Contact Information

The following project manager was associated with this project:

Kimberly Espinoza, Project Manager
kimberly.espinoza@trailofbits.com

The following engineering director was associated with this project:

Benjamin Samuels, Engineering Director, Blockchain
benjamin.samuels@trailofbits.com

The following consultant was associated with this project:

Guillermo Larregay, Consultant
guillermo.larregay@trailofbits.com

Project Timeline

The significant events and milestones of the project are listed below.

Date	Event
June 9, 2026	Pre-project kickoff call
June 18, 2026	Delivery of report draft
June 22, 2026	Report readout meeting
July 1, 2026	Completion of fix review
July 14, 2026	Delivery of final comprehensive report

Executive Summary

Engagement Overview

Gensyn engaged Trail of Bits to review the security of the automated settlement and permit implementations.

The code implements the automated settlement and permit changes for Gensyn Delphi's dynamic parimutuel prediction markets. The main components are the market factory, the dynamic parimutuel gateway, the market implementation, and the Truebit oracle relayer.

Automated settlement allows a market to be resolved without relying on a person to manually choose the result. Any account can trigger the settlement process, an external oracle system evaluates the market information, and the contracts use that response to either finalize the winning outcome or mark the market as unresolved if the oracle cannot provide a valid answer. Once that happens, users can recover value according to the market's final state.

The permit feature defined in ERC-2612 lets users authorize token spending and place trades in a single transaction, without the need for submitting a separate token approval transaction.

A consultant conducted the review from June 12 to June 18, 2026, for a total of one engineer-week of effort. Our testing efforts focused on the automated settlement flow, oracle relayer integration, market lifecycle transitions, fee accounting, access-control boundaries, and the permit-based trading path. With full access to source code and documentation, we performed static and dynamic testing of the code in scope, using automated and manual processes.

Observations and Impact

The review identified a recurring pattern around automated settlement lifecycle coordination and trust boundaries. The most important issues are that terminal market behavior can be blocked by recipient transfer failures (TOB-GEN-AS-1); the oracle relayer can finalize markets without a preceding resolution request (TOB-GEN-AS-2); and in-flight markets can be stranded by relayer rotation or duplicate WatchTower execution IDs (TOB-GEN-AS-4, TOB-GEN-AS-5). These issues can prevent markets from reaching the intended outcome-based settlement path, force users into liquidation instead of redemption, or make market funds inaccessible in specific terminal-state scenarios.

The review also found that permissionless market creation interacts with the off-chain settlement backend in ways that should be explicitly constrained or documented (TOB-GEN-AS-3), and that the permit-based buy flow is vulnerable to a standard consumed-nonce race that can cause user transactions to revert (TOB-GEN-AS-6). The codebase has clear component separation, explicit access-control modifiers, meaningful

automated settlement tests, and useful event coverage for the new flow; strengthening request-state invariants, relayer lifecycle handling, metadata validation, and operational monitoring would improve the reliability of the automated settlement design.

Recommendations

Based on the codebase maturity evaluation and findings identified during the security review, Trail of Bits recommends that Gensyn take the following steps:

- **Remediate the findings disclosed in this report.** These findings affect automated settlement reliability, market exit availability, oracle request handling, metadata trust boundaries, and permit-based trading. They should be addressed through direct fixes or broader refactoring efforts.
- **Strengthen the automated settlement lifecycle and improve the terminal-state fund distribution.** Ensure that settlement outcomes are tied to valid resolution requests, that relayer changes do not strand active markets, and that oracle responses cannot overwrite or invalidate existing requests. Use payout patterns that prevent a single recipient from blocking settlement, liquidation, or user exits.
- **Clarify and enforce the metadata trust model.** Ensure that market metadata used by automated settlement comes from approved sources or is validated before it is used.
- **Expand regression and invariant testing.** Add tests for the reported findings and for settlement lifecycle edge cases so future changes preserve the intended behavior. Add monitoring and response procedures for failed callbacks, unresolved settlement requests, relayer changes, unexpected terminal events, and privileged configuration changes.

Finding Severities and Categories

The following tables provide the number of findings by severity and category.

EXPOSURE ANALYSIS

Severity	*Count*
High	1
Medium	1
Low	4
Informational	0
Undetermined	0

CATEGORY BREAKDOWN

Category	*Count*
Access Controls	1
Data Validation	2
Denial of Service	1
Timing	2

Project Goals

The engagement was scoped to provide a security assessment of the Gensyn Automated Settlement and Permit. Specifically, we sought to answer the following non-exhaustive list of questions:

- Does the permissionless `resolveMarket` flow correctly pay keeper incentives without enabling double resolution, premature settlement, or fund loss?
- Are keeper fees and oracle fees reserved, disbursed, refunded, and accounted for correctly across successful settlement, oracle failure, market expiry, and liquidation?
- Does the Truebit relayer correctly validate WatchTower callbacks, execution statuses, result encoding, outcome bounds, and replay attempts?
- Are the gateway owner, relayer owner, WatchTower, keeper, market creator, trader, and fee-recipient roles separated and constrained appropriately?
- Can changing the oracle relayer through `setOracleRelayer` break in-flight markets, bypass intended settlement flow, or create overlapping trust assumptions?
- Does the ERC-2612 permit integration preserve user slippage limits, nonce semantics, approval scope, and trade execution assumptions?
- Are market metadata URIs and content hashes used consistently with the intended off-chain settlement trust model?
- Do tests and invariants cover the new automated-settlement and permit behavior, including nonzero keeper and oracle fee configurations?
- Are the documented trust assumptions, user-facing settlement behavior, and operational recommendations consistent with the implemented contracts?

Project Targets

The engagement involved reviewing and testing the following target.

automated-settlement branch

Repository	https://github.com/gensyn-ai/node/
Version	Commit `953295a`
Type	Solidity
Platform	EVM

Additionally, on June 17, 2026, the following pull requests were added to the targets:

PR #5162

Repository	https://github.com/gensyn-ai/node/
Version	Commit `6ce0a86`
Type	Solidity
Platform	EVM

PR #5170

Repository	https://github.com/gensyn-ai/node/
Version	Commit `571f07d`
Type	Solidity
Platform	EVM

PR #5171

Repository	https://github.com/gensyn-ai/node/
Version	Commit `a81060c`
Type	Solidity
Platform	EVM

For the fix review performed on July 1, 2026, the repository branches were rebased and some of the commits referenced above changed when merged. More information about the fix review can be found in appendix D, and the SHA-256 hashes of the in-scope files that were considered are provided below:

```
src/delphi/IOracle.sol
              04dd4d799286a597c1476b67c7727f388265957c7243cfc3fe940da4075dbe29

src/delphi/dynamicParimutuel/gateway/DynamicParimutuelGateway.sol
              cd840c30421a6914c623e3d3c6e21bc6453953492604f8f01657a9596df83675

src/delphi/dynamicParimutuel/gateway/IDynamicParimutuelGateway.sol
              1ab31feffcc058e691db61a505c6d1ce17c97a86fc1277777fdb4c8ec58b2676

src/delphi/dynamicParimutuel/gateway/IDynamicParimutuelGatewayErrors.sol
              48de06758c70427b35a326f522ad758efd6962531b4a74bf45984289a9c2cd34

src/delphi/dynamicParimutuel/implementation/DynamicParimutuelMarket.sol
              63d0e48e3eda9ef90eb0527965131aa8b8a5b51afbea4d9468aeec759ecaa2fb

src/delphi/dynamicParimutuel/implementation/IDynamicParimutuelMarket.sol
              bf5be0ff7ac58b226794690502559d809f5f45179383243f7aff57a535626493

src/delphi/dynamicParimutuel/implementation/IDynamicParimutuelMarketErrors.sol
              d3d29ebdbc3768fbb408afa466d81915cf2fad3facca5588ecb135c7054cdfa1

src/delphi/dynamicParimutuel/implementation/IDynamicParimutuelMarketTypes.sol
              652a928de98136f329b3840a2b900977fd27f239caae5e1e276331aa258d4890

src/delphi/factory/DelphiFactory.sol
              deebf13d25f380edfeab7e66bf972a990360f3a1c438c809d43eda4828e285b6

src/delphi/factory/IDelphiFactory.sol
              9380b1cf6f2f4e6d29ed20752481ab588525d5e488d2d118000e3e685506650b

src/delphi/factory/IDelphiFactoryErrors.sol
              dba42077d1cf17ae3a5b1b3f1ce4bb4125e5818a7b0c07204ef7a417e7d5d32f

src/mock/MockToken.sol
              9f2a7749f9277fc7f095e7efb78f02b414767056c0cf22f0cf5cc3f68c13ff67

src/delphi/oracle/TruebitOracleRelayer.sol
              174f624c660fed3508f4fb1361aa699d4f56fbdb666470742dc3b427058ba22e

src/delphi/oracle/truebit/abstract/Types.sol
              56001280c9553de38d09e555466da1a432ff1a9ea37895a0312eae4540c63c28

src/delphi/oracle/truebit/interfaces/IBaseTBContract.sol
              0075fbc167b5343867e56f73d4b36b7ea9f4303e53db5d24f893f57b154a58da
```

```
src/delphi/oracle/truebit/interfaces/IWatchTower.sol
              3fee9d5e2eda96dfdef8fe9cd08c014b86eb3d5c2bff77b2685d68c759e08fbf
```

Project Coverage

This section provides an overview of the analysis coverage of the review, as determined by our high-level engagement goals. Our approaches included the following:

- Review of the automated settlement lifecycle, including how markets move through resolution, settlement, failure, expiry, redemption, and liquidation, and whether those paths remain mutually exclusive
- Manual analysis of the gateway, market, factory, and oracle relayer contracts to identify authorization gaps, lifecycle bypasses, unsafe external calls, and cross-contract trust assumptions
- Review of the Truebit integration, including request-body construction, WatchTower callback validation, execution ID tracking, replay protection, malformed-result handling, and failure behavior
- Analysis of keeper and oracle fee accounting across market creation, permissionless resolution, successful settlement, oracle failure, expiry, and liquidation
- Review of ERC-2612 permit behavior and market metadata handling in the automated settlement path

Coverage Limitations

Because of the time-boxed nature of testing work, it is common to encounter coverage limitations. The following list outlines the coverage limitations of the engagement and indicates system elements that may warrant further review:

- The review focused on the automated settlement and permit changes introduced between the `main` and `automated-settlement` branches, limited to the scoped smart-contract files: `DelphiFactory.sol`, `DynamicParimutuelGateway.sol`, `DynamicParimutuelMarket.sol`, `DynamicParimutuelMath.sol`, `TruebitOracleRelayer.sol`, the related Delphi gateway/factory/market interfaces and errors, `IOracle.sol`, and `MockToken.sol` for ERC-2612 permit support.
- The scope also included directly related smart-contract tests, deployment scripts, Foundry configuration, and documentation needed to evaluate the scoped changes, but did not extend to a full review of the broader Gensyn monorepo.
- External libraries, generated artifacts, vendored dependencies, unrelated smart-contract components, and non-smart-contract services were considered to be out of scope.

- We covered the on-chain Truebit relayer integration, but not the Truebit backend, WatchTower deployment, API task implementation, or other off-chain settlement infrastructure.

Codebase Maturity Evaluation

Trail of Bits uses a traffic-light protocol to provide each client with a clear understanding of the areas in which its codebase is mature, immature, or underdeveloped. Deficiencies identified here often stem from root causes within the software development life cycle that should be addressed through standardization measures (e.g., the use of common libraries, functions, or frameworks) or training and awareness programs.

Category	Summary	Result
Arithmetic	The arithmetic is well documented and bounded. Solidity 0.8.30 provides checked arithmetic; the dynamic parimutuel math library documents rounding direction against the user; and the settlement-fee changes preserve the existing token-decimal convention. The factory also enforces that settlement fees do not exceed the market-creation fee before any market can be deployed from the implementation.	**Satisfactory**
Auditing	The contracts emit events for the new operational and terminal-state paths, including relayer changes, resolution requests, settlement, failure, relayer request IDs, callback failures, and oracle-fee-recipient changes. Some of the issues found can alter the event emission sequence: TOB-GEN-AS-2 can produce events with no preceding request; TOB-GEN-AS-4 can produce a relayer-side request with no terminal event; and TOB-GEN-AS-5 can produce events with duplicate WatchTower execution IDs. At the time of the audit, we were unaware of an off-chain monitoring infrastructure or an incident response plan.	**Moderate**
Authentication / Access Controls	The code has a clear access control structure. The gateway restricts relayer rotation to the owner. The registered relayer is the only caller of `settleMarket` and `failMarket`. The relayer restricts request creation to the gateway, callback delivery to the WatchTower address, upgrades to the relayer owner, and oracle-fee-recipient changes to the relayer owner. The market preserves `onlyGateway` on the new state-changing methods. Ownership transfers are handled using two-step procedures.	**Moderate**

	However, gaps in the access controls still exist: TOB-GEN-AS-2 shows that the registered relayer can settle or fail markets without a preceding request because `settleMarket` and `failMarket` do not check `settlementLocked`, while TOB-GEN-AS-1 shows that a permissionlessly supplied market creator can block all exits if token transfers to that address revert.	
Complexity Management	The implementation is organized into clear contract roles: the factory clones markets and forwards fees, the gateway remains the trading and settlement surface, the market implementation performs accounting and lifecycle transitions, and the relayer encapsulates the external oracle request and callback flow. The new relayer is sectioned by constants, storage, errors, events, modifiers, lifecycle, owner, getter, and internal logic, and the most important functions are flat enough to review manually. Responsibilities are separated cleanly, and custom errors and interfaces are used throughout the code.	**Satisfactory**
Cryptography and Key Management	No custom cryptography or key management implementations are in scope.	**Not Applicable**
Decentralization	The system relies on centralized operational actions and privileged actors that are controlled by Gensyn. These administrative actions are executed immediately, with no timelocks. The privileged accounts are documented to be multisignature wallets; however, there is no mention about the number of signers and threshold used. In terms of upgradeability, the deployed markets are non-upgradeable EIP-1167 clones, the gateway is not upgradeable, and the relayer is UUPS-upgradeable to allow oracle integration changes. The codebase would benefit from documenting production ownership, relayer-upgrade authority, relayer-rotation procedures, and any timelock or multisig controls used for privileged actions. It would also benefit from a relayer-rotation design that accounts for in-flight requests and a terminal-exit design that isolates untrusted recipient behavior.	**Moderate**

Documentation	The codebase includes developer-facing documentation in the form of NatSpec and inline comments around the automated settlement flow, oracle relayer callbacks, fee handling, and liquidation behavior. The project also maintains external user-facing documentation, but its alignment with the current automated settlement implementation was not verified as part of this review. The documentation would benefit from a synchronized developer overview of the current automated settlement design, including `resolveMarket`, relayer callbacks, `settleMarket`, `failMarket`, the oracle and keeper fee behavior, and relayer rotation. The external user documentation should also be checked and updated where needed so that user-facing settlement behavior matches the deployed contract flow.	**Moderate**
Low-Level Manipulation	The only low-level code in the scope is related to ERC-7201 upgradeable storage.	**Not Applicable**
Testing and Verification	The new code in the automated-settlement branch adds substantial tests. A significant amount of unit, integration, fuzzing, and fork tests cover protocol functionality. Testing is part of the CI pipeline. However, the reported issues show room for improvement in the existing test suite. It can be improved by adding new tests for adversarial cases to avoid regressions in the future, and expanding invariant coverage. We recommend implementing mutation testing strategies to measure and improve the coverage of the current test suite.	**Moderate**
Transaction Ordering	Slippage protections are in place for market operations. However, front-running risk exists where an attacker can grief a legitimate user by sending a signed permit transaction. This category can be improved by implementing a consumed-permit fallback for bundled permit-and-buy transactions (TOB-GEN-AS-6) and a relayer-rotation process that preserves in-flight callbacks (TOB-GEN-AS-4).	**Moderate**

Summary of Findings

The table below summarizes the findings of the review, including details on type and severity.

ID	Title	Type	Severity
1	Blacklisted market creator address permanently locks market funds	Denial of Service	High
2	Oracle relayer can settle or fail markets without a preceding resolution request	Access Controls	Medium
3	Permissionless market creators can send untrusted metadata references to the Truebit backend	Data Validation	Low
4	In-flight markets are stranded when the gateway oracle relayer is unset or rotated mid-flight	Timing	Low
5	Malicious WatchTower can orphan an in-flight market by returning a duplicate execution ID	Data Validation	Low
6	Front-running the permit in buyExactOutWithPermit causes the bundled buy to revert	Timing	Low

Detailed Findings

1. Blacklisted market creator address permanently locks market funds	
Severity: **High**	Difficulty: **Medium**
Type: Denial of Service	Finding ID: TOB-GEN-AS-1
Target: `delphi/smart-contracts/src/delphi/dynamicParimutuel/implementation/DynamicParimutuelMarket.sol`	

Description

Both the settlement and liquidation paths of a `DynamicParimutuelMarket` push USDC to the market creator via unconditional `safeTransfer` calls. The market creator address is set to `msg.sender` of `DelphiFactory.deployNewMarketProxy` at creation time, is permanently immutable thereafter, and is untrusted because the factory is permissionless. If the address ever causes USDC's `transfer` to revert (e.g., by being added to USDC's blocklist after the market is created), every terminal exit path reverts at one of those transfers, and the pool of trader assets, accrued trading fees, and any settlement fees still held by the market becomes inaccessible. The protocol exposes no mechanism to bypass, redirect, or escrow the creator-side payment.

The `settleMarket` function transfers to the creator once, in a single call that aggregates the creator's outcome reward, the refund, and the creator's cut of trading fees:

```
// Interactions: Give market creator tokens
uint256 marketCreatorTotal = _marketCreatorReward + _refund +
_marketCreatorTradingFeesCut;
TOKEN.safeTransfer(marketCreator, marketCreatorTotal);
```

Figure 1.1: The creator-side transfer in `settleMarket` (node/delphi/smart-contracts/src/delphi/dynamicParimutuel/implementation/DynamicParimutuelMarket.sol#L364–L366)

The `_liquidateMarketCreationShares` function, called as the first step of every `liquidate` and `liquidateMarketCreationShares` invocation on a market in `EXPIRED` or `FAILED` status, transfers to the creator up to three additional times in a single call: one for the liquidation value plus refund plus creator fee cut, one for the returned oracle fee, and one for the returned keeper fee.

```
uint256 marketCreatorTotal = liquidationValue + refund +
marketCreatorTradingFeesCut;
if (marketCreatorTotal > 0) {
    TOKEN.safeTransfer(marketCreator, marketCreatorTotal);
}
if (ORACLE_FEE > 0) {
    TOKEN.safeTransfer(marketCreator, ORACLE_FEE);
}

// ...

uint256 keeperFeeReturned = 0;
if (!keeperFeePaid && KEEPER_FEE > 0) {
    keeperFeeReturned = KEEPER_FEE;
    TOKEN.safeTransfer(marketCreator, KEEPER_FEE);
}
```

Figure 1.2: The three creator-side transfers in `_liquidateMarketCreationShares` (node/delphi/smart-contracts/src/delphi/dynamicParimutuel/implementation/DynamicParimutuelMarket.sol#L756–L774)

A blacklisted creator address, therefore, denies every terminal exit. When `settleMarket` reverts, the market remains in `AWAITING_SETTLEMENT` until the settlement deadline elapses, at which point its status transitions to `EXPIRED`. The only remaining path out of the pool is `liquidate`, and `liquidate` enters `_liquidateMarketCreationShares` first; that call reverts on the same creator transfer, and every subsequent attempt reverts at the same point. The pool of trader collateral, accrued trading fees, and any settlement fees still held by the market remain inaccessible indefinitely.

The same terminal transitions also push funds to `TRADING_FEES_RECIPIENT` and `oracleFeeRecipient`, but those recipients have different trust properties from `marketCreator`. A blocked `TRADING_FEES_RECIPIENT` can also cause `settleMarket` to revert, and can cause liquidation to revert when a nonzero trading-fee recipient cut is due; however, that address is an immutable protocol-selected treasury for the market implementation rather than a permissionlessly supplied market-creator address, and is therefore less likely to become blocklisted. `oracleFeeRecipient` is mutable on the relayer through `setOracleFeeRecipient`, gated by the relayer owner, and is read live on every callback; the owner can rotate it in a single transaction if it is ever blocklisted, and the `EXPIRED` liquidation path does not transfer to it at all (the returned oracle fee is sent to the market creator), so a blocked `oracleFeeRecipient` produces at most a transient settlement failure that resolves through natural expiry.

Exploit Scenario

A user creates a market through `DelphiFactory.deployNewMarketProxy` and becomes the `marketCreator`. Trading proceeds normally for the full trading window; buyers and sellers transact, and fees accrue USDC collateral into the pool. After the market is created, USDC's issuer adds the `marketCreator` address to the token's sanctions blocklist as part

of a compliance enforcement action unrelated to the protocol. The trading deadline passes, the keeper calls `resolveMarket`, and the keeper fee is paid out atomically with settlement being locked. The oracle then delivers the callback into `settleMarket`; the call reverts on the USDC transfer to the now-blocklisted `marketCreator`, and the market remains in `AWAITING_SETTLEMENT` until the settlement deadline elapses and it progresses to `EXPIRED`. Any trader then calls `liquidate`, which also reverts at the same transfer. The entire pool of trader collateral, the accrued trading fees, and the reserved oracle fee remain permanently inaccessible.

Recommendations

Short term, replace the push payments to the market creator with a pull payment credit. Track the amount owed to the creator in a per-market mapping, replace the current `safeTransfer` calls at the three creator-side sites (figures 1.1 and 1.2), and expose a function that lets the creator withdraw the accumulated balance. This way, the terminal-state transitions complete regardless of whether the creator's address can receive tokens at that moment.

Long term, adopt pull payment accounting as a project-wide invariant for any contract that distributes funds to a recipient supplied by untrusted input. Add an invariant test that fuzzes recipient behavior across every terminal transition, including blocklisted, reverting, and gas-grief recipients, and treat any push-payment to an untrusted-input address as an issue during code review.

2. Oracle relayer can settle or fail markets without a preceding resolution request

Severity: **Medium**	Difficulty: **High**
Type: Access Controls	Finding ID: TOB-GEN-AS-2

Target: `delphi/smart-contracts/src/delphi/dynamicParimutuel/gateway/DynamicParimutuelGateway.sol`

Description

The gateway does not require a market to have a gateway-issued resolution request before it accepts terminal-state calls from the registered oracle relayer. A compromised or malicious registered relayer can therefore settle or fail any factory-deployed market that is currently in `AWAITING_SETTLEMENT`, even when no keeper has called `resolveMarket`, no WatchTower request exists, and no relayer-side `pendingRequests` entry can be correlated to the market.

The intended request path sets `settlementLocked[marketProxy] = true`, pays the keeper fee through the market, calls the relayer, and emits `MarketResolutionRequested`.

```
function resolveMarket(address marketProxy) external
ifDeployedByFactory(IDynamicParimutuelMarket(marketProxy)) {
    if (oracleRelayer == address(0)) revert OracleRelayerNotSet();
    if (settlementLocked[marketProxy]) revert SettlementAlreadyLocked(marketProxy);

    // Effects: Lock settlement before calling out
    settlementLocked[marketProxy] = true;
```

Figure 2.1: The request path sets `settlementLocked` before dispatching to the relayer. (node/delphi/smart-contracts/src/delphi/dynamicParimutuel/gateway/DynamicParimutuelGateway.sol#L199–L204)

The terminal paths do not consume or check that flag. They check only that the caller is the current `oracleRelayer` and that the target market was deployed by the factory.

```
function settleMarket(address marketProxy, uint256 winningOutcomeIdx, address
oracleFeeRecipient)
    external
    onlyOracleRelayer
    ifDeployedByFactory(IDynamicParimutuelMarket(marketProxy))
{
```

```
    // Interactions: Settle the market and transfer the oracle fee atomically
(folded into settleMarket)
    IDynamicParimutuelMarket(marketProxy).settleMarket(winningOutcomeIdx,
oracleFeeRecipient);

    emit GatewayMarketSettled(marketProxy, winningOutcomeIdx);
}
```

Figure 2.2: `settleMarket` does not require `settlementLocked[marketProxy]` to be true. (node/delphi/smart-contracts/src/delphi/dynamicParimutuel/gateway/DynamicParimutuelGateway.sol#L220–L229)

```
function failMarket(address marketProxy)
    external
    onlyOracleRelayer
    ifDeployedByFactory(IDynamicParimutuelMarket(marketProxy))
{
    // Interactions: Fail the market. Oracle fee is returned to the creator inside
    // _liquidateMarketCreationShares() — the single guaranteed execution point for
    // both FAILED and EXPIRED paths — not here, to avoid double payment.
    IDynamicParimutuelMarket(marketProxy).failMarket();

    emit GatewayMarketFailed(marketProxy);
}
```

Figure 2.3: `failMarket` has the same missing precondition. (node/delphi/smart-contracts/src/delphi/dynamicParimutuel/gateway/DynamicParimutuelGateway.sol#L232–L243)

The market contract limits the blast radius to markets in the settlement window. Both terminal functions can be reached only through the gateway and require `MarketStatus.AWAITING_SETTLEMENT`, so the bypass cannot affect markets that are still open, already settled, failed, or expired.

However, that market-level status guard does not prove that the gateway dispatched a request. A direct `settleMarket` call lets the relayer choose `winningOutcomeIdx` and `oracleFeeRecipient`, pays the oracle fee to that recipient, and leaves the keeper fee reserved in the market because `transferKeeperFee` never ran and settled markets cannot enter `_liquidateMarketCreationShares`. A direct `failMarket` call does not pay the oracle fee to the relayer, but it moves the market to `FAILED` without an oracle request; users then exit through pro-rata liquidation instead of outcome-based redemption, and the oracle and unpaid keeper fees are returned through the liquidation path.

The issue is not reachable by an arbitrary external account; the caller must be the registered relayer. Also, under the current trust assumptions, the relayer could return a malicious result even for a legitimate request. The missing `settlementLocked` check

broadens that trust assumption by allowing the relayer to create a terminal state for markets with no corresponding gateway request.

Exploit Scenario

A compromised or malicious registered relayer calls `gateway.settleMarket(marketProxy, attackerChosenOutcome, attackerRecipient)` directly after a target market passes its trading deadline and enters `AWAITING_SETTLEMENT`, without any keeper calling `resolveMarket`. The gateway accepts the call because the caller is the current relayer, and the market accepts it because the market is in the settlement window. The market transitions to SETTLED with the attacker's chosen outcome, the oracle fee is paid to the attacker's recipient, the keeper fee remains stranded in the market, and off-chain observers see a `GatewayMarketSettled` with no preceding `MarketResolutionRequested` event for that market.

Recommendations

Short term, add a request-state precondition to `settleMarket` and `failMarket`. Both functions should revert before calling the market unless `settlementLocked[market]` is true. The relayer must not finalize a market until the gateway has accepted a `resolveMarket` call for that market, set the settlement lock, and paid the keeper fee through the documented request path.

Long term, treat "no terminal market transition without a gateway-issued resolution request" as a protocol invariant. Document this invariant in the gateway and market specifications, and add invariant tests that fuzz call ordering, caller identity, market status, and relayer behavior to ensure `settleMarket` and `failMarket` cannot execute unless `resolveMarket` has already locked the market.

3. Permissionless market creators can send untrusted metadata references to the Truebit backend

Severity: **Low**	Difficulty: **Low**
Type: Data Validation	Finding ID: TOB-GEN-AS-3

Target: `delphi/smart-contracts/src/delphi/oracle/TruebitOracleRelayer.sol`, `delphi/smart-contracts/src/delphi/dynamicParimutuel/gateway/DynamicParimutuelGateway.sol`, `delphi/smart-contracts/src/delphi/factory/DelphiFactory.sol`, `delphi/smart-contracts/src/delphi/dynamicParimutuel/implementation/DynamicParimutuelMarket.sol`

Description

The automated settlement path accepts metadata references from any factory-deployed market and forwards the URI to the Truebit backend. Because market creation is permissionless and the contracts do not enforce the trusted private-GCS URI assumption, an untrusted market creator can cause attacker-controlled, malformed, non-GCS, or otherwise unsupported metadata references to reach the backend through the normal settlement flow. If the backend dereferences the URI, it will make an outbound request to an attacker-selected endpoint and receive attacker-provided response data; if it parses the request body as JSON, it may also receive malformed or duplicate-keyed structured input because the URI is inserted into the body without escaping.

The contract-level market creation path is permissionless. Any account that can satisfy the deposit and configuration requirements can call `deployNewMarketProxy`, become the market creator, and choose the metadata URI and hash pair passed into the new market.

```
// Permissionless
function deployNewMarketProxy(
    uint256 initialDeposit_,
    IDelphiMarket.VerifiableUri calldata newMarketMetadata_,
    bytes calldata newMarketInitializationCalldata_
) external returns (address newMarketProxy) {
    // Interactions: Deploy new market proxy
    newMarketProxy = IMPLEMENTATION.clone();
```

Figure 3.1: The factory exposes a permissionless market creation path. (node/delphi/smart-contracts/src/delphi/factory/DelphiFactory.sol#L95–L102)

Market metadata is stored on-chain as a URI plus a `uriContentHash` commitment. The hash gives off-chain consumers a way to verify that the bytes fetched from the URI are the same bytes committed at market creation. However, the contract validates only that both fields are present. It does not require the URI to point to a trusted host, a private GCS bucket, or a creator controlled by the protocol.

```
function _validateVerifiableUri(VerifiableUri calldata verifiableUri) internal pure
{
    if (bytes(verifiableUri.uri).length == 0) {
        revert EmptyUri();
    }
    if (verifiableUri.uriContentHash == bytes32(0)) {
        revert EmptyUriContentHash();
    }
}
```

Figure 3.2: Metadata validation requires only a non-empty URI and a nonzero hash. (node/delphi/smart-contracts/src/delphi/dynamicParimutuel/implementation/DynamicParimutuelMarket.sol#L861–L868)

The gateway can dispatch any factory-deployed market to the oracle relayer once that market reaches the settlement path. It does not distinguish trusted market creators from untrusted market creators. The relayer then constructs the WatchTower task body through raw string concatenation, sending only `metadata_path`, `created_at`, and `resolves_at`. The stored `uriContentHash` is not included in the request body, and the creator-provided URI is not escaped before insertion into the JSON string.

```
function resolveMarket(address marketProxy) external override onlyGateway {
    IDynamicParimutuelMarket market = IDynamicParimutuelMarket(marketProxy);
    string memory body = string.concat(
        '{"metadata_path": "',
        market.getMarketMetadata().uri,
        '", "created_at": ',
        Strings.toString(market.createdAt()),
        ', "resolves_at": ',
        Strings.toString(market.getMarket().config.tradingDeadline),
        "}"
    );
```

Figure 3.3: The Truebit task body inserts the metadata URI through raw string concatenation and omits the metadata content hash. (node/delphi/smart-contracts/src/delphi/oracle/TruebitOracleRelayer.sol#L149–L159)

Under the Gensyn-stated trust model, market metadata URIs point to private GCS buckets controlled by Gensyn and are therefore trusted by the automated settlement backend. That assumption is not enforced by the in-scope contracts: the factory is permissionless, the gateway accepts any factory-deployed market, and neither the factory nor the market

enforces that the URI belongs to a trusted origin. An attacker who creates a market directly can therefore supply a URI that, if fetched, exposes backend egress metadata such as the source IP address to the attacker-controlled endpoint. The attacker can also include characters such as quotation marks, reverse solidus characters, control characters, or JSON structural bytes in the URI; these characters are accepted by the market contract and can make the relayer's task body malformed or parser-dependent.

The Truebit task implementation and associated backend are outside the scope of this review, so we did not assess how the backend handles untrusted, malformed, or non-GCS metadata references.

Exploit Scenario

An attacker deploys a market directly through the factory with a metadata URI pointing to an attacker-controlled HTTPS endpoint, optionally including JSON-control characters in the URI. After the market reaches the settlement window, the attacker or another keeper calls the permissionless resolution path. The gateway accepts the factory-deployed market, and the relayer sends the attacker-controlled URI to the Truebit backend as `metadata_path`. If the backend dereferences the URI, the attacker's endpoint observes backend egress metadata such as the source IP address and returns arbitrary response bytes to the backend. If the backend parses the relayer body as JSON, the unescaped URI may also produce malformed or duplicate-keyed structured input. Further effects, including parser failures, prompt manipulation, logging issues, or settlement behavior, depend on the out-of-scope Truebit backend implementation and were not verified during this review.

Recommendations

Short term, prevent automated settlement from dispatching untrusted metadata references to the Truebit backend. Either enforce the trusted-GCS URI policy before a market can be resolved, require the gateway or relayer to recognize only markets created through an approved creation path, or include `uriContentHash` in the Truebit request body and require the off-chain task to validate the fetched metadata before parsing or using it. Also reject URI values containing quotation marks, reverse solidus characters, control characters, or other bytes that are unsafe in the current JSON string position.

Long term, document and enforce a single trust model for market metadata. If only trusted creators and private GCS-hosted metadata are supported, enforce that boundary in the in-scope creation or resolution path. If permissionless market creation is intended, require the automated settlement flow to validate `uriContentHash` before using market metadata.

4. In-flight markets are stranded when the gateway oracle relayer is unset or rotated mid-flight

Severity: **Low**	Difficulty: **High**
Type: Timing	Finding ID: TOB-GEN-AS-4
Target: `delphi/smart-contracts/src/delphi/dynamicParimutuel/gateway/DynamicParimutuelGateway.sol`, `delphi/smart-contracts/src/delphi/factory/DelphiFactory.sol`	

Description

The gateway's `oracleRelayer` state is not coordinated with the in-flight resolution lifecycle. Two distinct operational events can drive markets into a stranded state: a market created before the owner has called `setOracleRelayer`, and a market whose resolution is in flight when the owner rotates `oracleRelayer` to a new address. In the first case, the market is recoverable only if the owner sets the relayer before `settlementDeadline`; in the second case, the in-scope contracts expose no recovery path once the callback to the original relayer has reverted. Otherwise, the affected market remains in `AWAITING_SETTLEMENT` until `settlementDeadline` elapses, at which point it progresses to `EXPIRED` and pays out through `liquidate` as a pro-rata refund rather than reaching the outcome-based settlement that the gateway's `resolveMarket` path otherwise produces.

The first event is the deployment race. `DelphiFactory.deployNewMarketProxy` is permissionless and does not consult the gateway's `oracleRelayer` state. A market created while `gateway.oracleRelayer == address(0)` opens for trading normally; when `tradingDeadline` elapses and a keeper attempts to call `resolveMarket`, the gateway reverts with `OracleRelayerNotSet`:

```
function resolveMarket(address marketProxy) external
ifDeployedByFactory(IDynamicParimutuelMarket(marketProxy)) {
    if (oracleRelayer == address(0)) revert OracleRelayerNotSet();
    if (settlementLocked[marketProxy]) revert SettlementAlreadyLocked(marketProxy);

    // Effects: Lock settlement before calling out
    settlementLocked[marketProxy] = true;
```

Figure 4.1: `Gateway.resolveMarket` reverts when the oracle relayer has not been wired, but the market was already allowed to be created.
(node/delphi/smart-contracts/src/delphi/dynamicParimutuel/gateway/DynamicParimutuelGateway.sol#L199–L204)

If the owner does not call `setOracleRelayer` before `settlementDeadline` elapses, the market reaches `EXPIRED` without a winning outcome ever being declared. If the owner does call `setOracleRelayer` within the settlement window, the next `resolveMarket` call succeeds and the market resolves through the ordinary path. The vulnerable window is therefore the interval between market creation and the first successful `setOracleRelayer` call, bounded above by the market's `settlementDeadline`. Per the configuration bounds documented in the project's agent guide, that interval can be as short as two minutes of trading plus one hour of settlement, which gives the gateway operator little time to react if a market is created immediately after the gateway is deployed.

The second event is the rotation race. `setOracleRelayer` is one-step, has no constraint on in-flight resolutions, and does not surface the in-flight state to the caller:

```
function setOracleRelayer(address oracleRelayer_) external onlyOwner {
    if (oracleRelayer_ == address(0)) revert ZeroOracleRelayerAddress();

    address previous = oracleRelayer;
    oracleRelayer = oracleRelayer_;

    emit OracleRelayerSet(previous, oracleRelayer_);
}
```

Figure 4.2: `Gateway.setOracleRelayer` rotates immediately, without reference to in-flight resolutions.
(node/delphi/smart-contracts/src/delphi/dynamicParimutuel/gateway/DynamicParimutuelGateway.sol#L189–L196)

When `resolveMarket` has already been called for a market, the relayer has stored a `pendingRequests` entry mapping the WatchTower execution ID to the market, and the WatchTower has queued the off-chain task. If the owner calls `setOracleRelayer` before the callback arrives, the WatchTower still delivers the callback to the original relayer's address, the original relayer's `callbackTask` calls `gateway.settleMarket`, and the gateway's `onlyOracleRelayer` check reverts because the caller is the previous relayer rather than the new one. A fresh `resolveMarket` cannot be issued against the new relayer either, because `settlementLocked[marketProxy]` is already true. Without an out-of-band WatchTower replay after the gateway owner rotates back to the original relayer, the market is stranded for the remainder of `settlementDeadline` and then progresses to `EXPIRED`.

The two subcases differ in their economic side effects. In the deployment-race subcase, `resolveMarket` never executes for the affected market, so `transferKeeperFee` is never called; `keeperFeePaid` stays false and the keeper fee is returned to the creator in `_liquidateMarketCreationShares` on `EXPIRED`. In the rotation-race subcase, `transferKeeperFee` ran when the original `resolveMarket` executed; `keeperFeePaid` is true, so the keeper-fee refund branch is skipped and the creator absorbs the cost of a

resolution that did not deliver. The two subcases also leave different forensic traces: the deployment-race subcase produces no `ResolutionRequested` event for the affected market, while the rotation-race subcase produces a `ResolutionRequested` event recorded by the previous relayer's address and no corresponding gateway terminal event.

Exploit Scenario

The gateway owner rotates `oracleRelayer` while a market resolution is already in flight. Before the rotation, a keeper called `resolveMarket`, the gateway set `settlementLocked[marketProxy] = true`, the market paid the keeper fee, and the previous relayer recorded the WatchTower execution ID in `pendingRequests`. The owner then calls `setOracleRelayer` to install the new relayer before the WatchTower callback arrives. When the callback reaches the previous relayer, its `callbackTask` attempts to call `gateway.settleMarket`, but the gateway rejects the call because `msg.sender` is no longer the registered relayer. The request remains associated with the previous relayer, the new relayer has no pending request for the market, and the gateway will not accept a fresh `resolveMarket` call because `settlementLocked[marketProxy]` is already true. The market remains unresolved until `settlementDeadline` elapses, then progresses to `EXPIRED`; traders recover only through pro-rata liquidation, and the creator absorbs the keeper fee paid for a resolution that did not complete.

Recommendations

Short term, close the deployment race on-chain and consider allowing both the previous and new oracle relayers to complete callbacks for a bounded grace period after rotation. Add a single check in `DynamicParimutuelMarket.initialize` that reverts when `IDynamicParimutuelGateway(GATEWAY).oracleRelayer() == address(0)`, so that any call to `DelphiFactory.deployNewMarketProxy` issued before the gateway is set reverts atomically with the deployment attempt rather than producing a stranded market. For the rotation race, update the relayer authorization check so that the previous relayer can complete already in-flight requests for a short, documented interval after `setOracleRelayer`, while the new relayer receives new requests. Limit the previous relayer's authority by time and, if feasible, by request state, so that old relayers do not retain broad settlement authority after rotation.

Long term, codify the relayer lifecycle as a protocol invariant and cover it with focused tests. The test suite should verify that markets cannot be created before the relayer is wired, that relayer rotation does not orphan in-flight requests, that the previous relayer cannot settle unrelated markets or act after the grace period, and that expired in-flight markets liquidate with correct keeper-fee accounting.

5. Malicious WatchTower can orphan an in-flight market by returning a duplicate execution ID

Severity: **Low**	Difficulty: **High**
Type: Data Validation	Finding ID: TOB-GEN-AS-5
Target: `delphi/smart-contracts/src/delphi/oracle/TruebitOracleRelayer.sol`	

Description

`TruebitOracleRelayer.resolveMarket` stores the mapping from execution ID to market proxy by writing `pendingRequests[wtExecutionId] = marketProxy` directly, with no check that the slot is empty. The value of `wtExecutionId` is the return of an external call into the WatchTower, which the relayer documents elsewhere on the same contract as a potentially malicious party. Therefore, a WatchTower that returns the same `wtExecutionId` for two distinct requests overwrites the first market's entry with the second, silently orphaning the first.

```
// Overwrite safety: the WatchTower assigns a unique, monotonically incrementing
wtExecutionId
// per request (its requestIdCounter never resets). Combined with settlementLocked
(a given
// market can only be requested once), pendingRequests entries can never collide.
TruebitOracleRelayerStorage storage $ = _getTruebitOracleRelayerStorage();
$.pendingRequests[wtExecutionId] = marketProxy;
emit ResolutionRequested(marketProxy, wtExecutionId);
```

Figure 5.1: The `pendingRequests` write trusts the WatchTower-supplied ID to be unique. (node/delphi/smart-contracts/src/delphi/oracle/TruebitOracleRelayer.sol#L174–L179)

A duplicate execution ID breaks the one-to-one correlation between a WatchTower request and the market that should receive its callback. The later request remains reachable through the overwritten slot, while the earlier market is left locked with no pending request entry that can complete it. The in-scope contracts expose no path that can reconstruct which execution ID originally belonged to the earlier market, and `settlementLocked` prevents a fresh `resolveMarket` request from recovering the situation.

Once a market has been orphaned, its lifecycle proceeds through the timestamp fallback. The market remains in `AWAITING_SETTLEMENT` until `settlementDeadline` elapses, at which point `marketStatus` returns `EXPIRED`. `liquidate` becomes the only exit, and users recover their pro-rata share of the pool. The protocol intends `EXPIRED` to be a recoverable

failure mode, and that property still holds; the realized economic effect is the difference between the SETTLED outcome the market would otherwise have reached and the pro-rata refund it now delivers.

The fix is a single read-before-write check at the recording site. Inserting `if ($.pendingRequests[wtExecutionId] != address(0)) revert DuplicateExecutionId(wtExecutionId);` before the assignment in figure 5.1 closes the path by reverting on a colliding ID. The fix is fully compatible with the honest WatchTower flow, which assigns monotonically increasing IDs that never collide by construction.

Exploit Scenario

A compromised or adversarial WatchTower first returns `wtExecutionId = 42` for a resolution request for `marketA`, causing the relayer to record `pendingRequests[42] = marketA`. The WatchTower later returns 42 again for a separate request for `marketB`, and the relayer's unconditional write replaces the first entry with `pendingRequests[42] = marketB`. The WatchTower then delivers a callback for ID 42; `callbackTask` looks up `marketB`, deletes the entry, and forwards the callback to the gateway for `marketB` as if no collision had occurred. `marketA` no longer has a pending request entry, and `settlementLocked[marketA]` prevents a fresh request. `marketA` remains in `AWAITING_SETTLEMENT` until `settlementDeadline` elapses and then progresses to `EXPIRED`, paying out through `liquidate` as a pro-rata refund instead of the outcome-based settlement it would otherwise have reached.

Recommendations

Short term, validate the WatchTower-supplied execution ID on-chain before recording it. Add a dedicated error and check at the recording site that reverts when `pendingRequests[wtExecutionId]` is already populated, before the write that would overwrite the existing entry.

Long term, document execution-ID uniqueness as a protocol invariant and cover it with regression and invariant tests. The tests should use a mock WatchTower that returns duplicate IDs and should assert that the second request reverts, the original `pendingRequests` entry remains intact, and the first market can still be completed.

6. Front-running the permit in buyExactOutWithPermit causes the bundled buy to revert

Severity: **Low**	Difficulty: **Low**
Type: Timing	Finding ID: TOB-GEN-AS-6
Target: `delphi/smart-contracts/src/delphi/dynamicParimutuel/gateway/DynamicParimutuelGateway.sol`	

Description

`buyExactOutWithPermit` calls the token's `permit` function directly inside the same transaction as the buy, with no fallback for the case where the permit has already been consumed. A user broadcasting a `buyExactOutWithPermit` transaction to a public mempool reveals the permit signature `(v, r, s)` along with the `deadline`, `value`, and intended `spender`. Any observer can extract these values, submit a stand-alone `permit` call to the token contract with higher priority gas, and consume the user's ERC-2612 nonce before the bundled transaction is included. The user's transaction then reverts when its internal `permit` call sees the now-incremented nonce, leaving them with a failed transaction and the gas cost of the revert, but with the allowance they signed for already in place on the spender.

```
function buyExactOutWithPermit(
    IDynamicParimutuelMarket marketProxy,
    uint256 outcomeIdx,
    uint256 sharesOut,
    uint256 maxTokensIn,
    uint256 deadline,
    uint8 v,
    bytes32 r,
    bytes32 s
) external ifDeployedByFactory(marketProxy) returns (uint256 tokensIn) {
    IERC20Permit(address(TOKEN))
        .permit({
        owner: msg.sender, spender: address(marketProxy), value: maxTokensIn,
deadline: deadline, v: v, r: r, s: s
    });

    return buyExactOut(marketProxy, outcomeIdx, sharesOut, maxTokensIn);
}
```

Figure 6.1: The `permit` call is not wrapped in any fallback that handles the case where the nonce was consumed by a front-run.

(*node/delphi/smart-contracts/src/delphi/dynamicParimutuel/gateway/DynamicParimutuelGateway.sol#L126–L142*)

The mechanism is the standard ERC-2612 public-mempool race. Permit signatures are bound to a per-owner nonce that increments on each successful `permit` call against the token; the same signature cannot be replayed twice. An adversary who submits the user's signature to the token contract directly succeeds because the signature is still valid at the moment of submission, and the only state change the adversary triggers is the nonce increment plus the allowance grant. The user's subsequent transaction, which also tries to call `permit` with the same signature, reverts because the nonce check now fails. The adversary pays the gas for one extra `permit` call; the user pays the gas for the reverted bundled transaction.

The user's recovery from a successful grief is to issue a plain `buyExactOut` transaction without the permit. The stand-alone `permit` call performed by the adversary already established the allowance on the market clone, so the recovery transaction succeeds without a fresh signature.

Exploit Scenario

A user broadcasts a `buyExactOutWithPermit` transaction to a public mempool, containing a valid permit signature `(v, r, s)` over `(maxTokensIn, deadline)` for a known market. A mempool observer extracts the signature fields from the pending transaction and submits a stand-alone `permit` call to the token contract carrying those same parameters and a higher priority fee. The observer's call is included first, the user's ERC-2612 nonce increments, and the allowance on the market clone is set to `maxTokensIn`. The user's bundled transaction is then included; its internal `permit` call sees the incremented nonce, rejects the signature, and reverts the entire transaction. The user pays the gas for the revert and has no shares from the intended buy.

Recommendations

Short term, wrap the `permit` call in a try-catch block that tolerates a consumed nonce. After the try-catch block, read the user's allowance against the market clone and proceed with the buy when it is at least `maxTokensIn`; revert with a dedicated error otherwise.

Long term, treat the consumed-permit path as a standard test case for any entrypoint that combines a permit with another action. Add a small regression test pattern that submits the permit first, then calls the bundled entrypoint and asserts that it proceeds only when the expected allowance is already present. This prevents future permit wrappers from reintroducing the same revert-on-consumed-nonce behavior without requiring a new approval mechanism.

A. Vulnerability Categories

The following tables describe the vulnerability categories, severity levels, and difficulty levels used in this document.

Vulnerability Categories	
Category	**Description**
Access Controls	Insufficient authorization or assessment of rights
Auditing and Logging	Insufficient auditing of actions or logging of problems
Authentication	Improper identification of users
Configuration	Misconfigured servers, devices, or software components
Cryptography	A breach of system confidentiality or integrity
Data Exposure	Exposure of sensitive information
Data Validation	Improper reliance on the structure or values of data
Denial of Service	A system failure with an availability impact
Error Reporting	Insecure or insufficient reporting of error conditions
Patching	Use of an outdated software package or library
Session Management	Improper identification of authenticated users
Testing	Insufficient test methodology or test coverage
Timing	Race conditions or other order-of-operations flaws
Undefined Behavior	Undefined behavior triggered within the system

Severity Levels	
Severity	**Description**
Informational	The issue does not pose an immediate risk but is relevant to security best practices.
Undetermined	The extent of the risk was not determined during this engagement.
Low	The risk is small or is not one the client has indicated is important.
Medium	User information is at risk; exploitation could pose reputational, legal, or moderate financial risks.
High	The flaw could affect numerous users and have serious reputational, legal, or financial implications.

Difficulty Levels	
Difficulty	**Description**
Not Applicable	This issue is of informational severity and does not pose an immediate risk, so difficulty does not apply.
Undetermined	The difficulty of exploitation was not determined during this engagement.
Low	The flaw is well known; public tools for its exploitation exist or can be scripted.
Medium	An attacker must write an exploit or will need in-depth knowledge of the system.
High	An attacker must have privileged access to the system, may need to know complex technical details, or must discover other weaknesses to exploit this issue.

B. Code Maturity Categories

The following tables describe the code maturity categories and rating criteria used in this document.

Code Maturity Categories	
Category	**Description**
Arithmetic	The proper use of mathematical operations and semantics
Auditing	The use of event auditing and logging to support monitoring
Authentication / Access Controls	The use of robust access controls to handle identification and authorization and to ensure safe interactions with the system
Complexity Management	The presence of clear structures designed to manage system complexity, including the separation of system logic into clearly defined functions
Cryptography and Key Management	The safe use of cryptographic primitives and functions, along with the presence of robust mechanisms for key generation and distribution
Decentralization	The presence of a decentralized governance structure for mitigating insider threats and managing risks posed by contract upgrades
Documentation	The presence of comprehensive and readable codebase documentation
Low-Level Manipulation	The justified use of inline assembly and low-level calls
Testing and Verification	The presence of robust testing procedures (e.g., unit tests, integration tests, and verification methods) and sufficient test coverage
Transaction Ordering	The system's resistance to transaction-ordering attacks

Rating Criteria	
Rating	**Description**
Strong	No issues were found, and the system exceeds industry standards.
Satisfactory	Minor issues were found, but the system is compliant with best practices.
Moderate	Some issues that may affect system safety were found.
Weak	Many issues that affect system safety were found.
Missing	A required component is missing, significantly affecting system safety.
Not Applicable	The category does not apply to this review.
Not Considered	The category was not considered in this review.
Further Investigation Required	Further investigation is required to reach a meaningful conclusion.

C. Code Quality Recommendations

This appendix contains recommendations for findings that do not have immediate or obvious security implications. However, addressing them may enhance the code's readability and may prevent the introduction of vulnerabilities in the future.

- **Document and test the expected behavior for late oracle callbacks.** A callback that arrives after the settlement deadline is handled as a failed settlement path rather than as a successful resolution, which appears to be a liveness tradeoff. Adding explicit documentation and a regression test for this timing boundary would make future changes less likely to alter the intended expiry behavior accidentally.
- **Validate the WatchTower address in the relayer constructor.** The deployment helper checks for a zero WatchTower address, but the contract constructor itself accepts the value without a local zero-address guard. Mirroring the deployment-script validation in the contract would keep the invariant close to the state it protects.
- **Validate reused relayer implementations against the deployment configuration.** The relayer deployment helper validates the configured WatchTower, gateway, execution timeout, owner, and fee recipient before deployment. However, when an existing relayer implementation address is supplied, the helper casts that address to `TruebitOracleRelayer` without checking that it contains code or that its constructor-set immutables match the same configuration. Adding checks such as `implementation.code.length > 0`, `impl.watchTower() == watchTower`, and similar checks for the other immutables would make implementation reuse safer and would reduce the chance of deploying a proxy against stale or misconfigured implementation bytecode.
- **Document whether zero market-creation fees are valid.** `DelphiFactory` permits a zero market-creation fee, while production settlement parameters are expected to cover keeper and oracle fees. Capturing the intended deployment policy would make it clearer whether zero-fee configurations are only for local development and tests.

D. Fix Review Results

When undertaking a fix review, Trail of Bits reviews the fixes implemented for issues identified in the original report. This work involves a review of specific areas of the source code and system configuration, not comprehensive analysis of the system.

On July 1, 2026, Trail of Bits reviewed the fixes and mitigations implemented by the Gensyn team for the issues identified in this report. We reviewed each fix to determine its effectiveness in resolving the associated issue. Gensyn provided commit `3bcd928` in the `automated-settlement-tob-fixes` branch of the repository, where all fixes are grouped by commits.

The reviewed changes in the prediction market smart contracts were accompanied by targeted unit tests, which we verified against the project's Foundry test suite. Five of the six issues were resolved through smart-contract changes; one was addressed by out-of-scope changes to the off-chain backend and was therefore not reviewed. The fixes are well scoped and address the root causes of the reported issues.

In addition to the fixes, the team provided complementary commits that address the code quality recommendations from appendix C, and additional changes that were not part of the recommendations in the present report, consisting of improvements to the event emission in the code, changes to configuration parameters and the Truebit task constants, and the addition of new tests.

In summary, of the six issues described in this report, Gensyn has resolved four issues, has partially resolved one issue, and has applied off-chain, out-of-scope changes to mitigate the remaining issue. For additional information, please see the Detailed Fix Review Results below.

ID	Title	Severity	Status
1	Blacklisted market creator address permanently locks market funds	High	Resolved
2	Oracle relayer can settle or fail markets without a preceding resolution request	Medium	Resolved
3	Permissionless market creators can send untrusted metadata references to the Truebit backend	Low	Undetermined

4	In-flight markets are stranded when the gateway oracle relayer is unset or rotated mid-flight	Low	Partially Resolved
5	Malicious WatchTower can orphan an in-flight market by returning a duplicate execution ID	Low	Resolved
6	Front-running the permit in buyExactOutWithPermit causes the bundled buy to revert	Low	Resolved

Detailed Fix Review Results

TOB-GEN-AS-1: Blacklisted market creator address permanently locks market funds
Resolved in commit 51b2bbe. The `_liquidateMarketCreationShares` call was removed from the liquidation path of the market. The creator must call the `liquidateMarketCreationShares` function manually to claim their assets. This prevents the funds from being locked if the creator is blacklisted, but does not implement the full pull pattern. Fee recipients are still being transferred assets, but those are considered to be trusted and unlikely to be blacklisted.

The client provided the following context for this finding's fix status:

> *This fix removes the lock vector for every trader with a minimal, storage-free change. It does not attempt to also preserve outcome-based settlement when a protocol recipient is blocked — that residual is documented and deferred.*
>
> *Fully closing the settlement-degradation residual requires either (a) a pull-payment pattern on `settleMarket`'s recipient transfers, or (b) the cheaper option of making `TRADING_FEES_RECIPIENT` rotatable — which alone neutralizes the only non-recoverable case. Tracked internally.*

TOB-GEN-AS-2: Oracle relayer can settle or fail markets without a preceding resolution request
Resolved in commit 87de8e0. Both market resolution functions (`settleMarket` and `failMarket`) now enforce a check that a prior resolution request exists. Delphi tests were modified to cover the changes.

TOB-GEN-AS-3: Permissionless market creators can send untrusted metadata references to the Truebit backend
Undetermined. Gensyn stated that three independent off-chain layers mitigate the issue. The changes to the backend are out of scope for this review; therefore, we did not review the fix for this particular issue.

The client provided the following context for this finding's fix status:

> *Acknowledged. Attack surface is mitigated off-chain; no on-chain fix required.*

TOB-GEN-AS-4: In-flight markets are stranded when the gateway oracle relayer is unset or rotated mid-flight
Partially resolved in commit 92ac7a8. The deployment race condition was fixed by requiring the oracle to be set by the time initialization happens. The oracle relayer change situation is acknowledged; no code changes were made to address it.

The client provided the following context for this finding's fix status:

This is accepted: address-swaps are rare, owner-triggered, and operationally controlled.

TOB-GEN-AS-5: Malicious WatchTower can orphan an in-flight market by returning a duplicate execution ID

Resolved in commit `e5b0baf`. The `executionId` is now verified to be empty in the pending requests mapping, reverting if it tries to overwrite a previous request. A new test was added with the malicious or buggy WatchTower behavior.

TOB-GEN-AS-6: Front-running the permit in buyExactOutWithPermit causes the bundled buy to revert

Resolved in commit `2be4fe6`. The `permit` call is now wrapped in a try-catch block avoiding the revert when the permit is already consumed. New tests were added for every case.

E. Fix Review Status Categories

The following table describes the statuses used to indicate whether an issue has been sufficiently addressed.

Fix Status	
Status	**Description**
Undetermined	The status of the issue was not determined during this engagement.
Unresolved	The issue persists and has not been resolved.
Partially Resolved	The issue persists but has been partially resolved.
Resolved	The issue has been sufficiently resolved.

About Trail of Bits

Founded in 2012 and headquartered in New York, Trail of Bits provides technical security assessment and advisory services to some of the world's most targeted organizations. We combine high-end security research with a real-world attacker mentality to reduce risk and fortify code. With 100+ employees around the globe, we've helped secure critical software elements that support billions of end users, including Kubernetes and the Linux kernel.

We maintain an exhaustive list of publications at https://github.com/trailofbits/publications, with links to papers, presentations, public audit reports, and podcast appearances.

In recent years, Trail of Bits consultants have showcased cutting-edge research through presentations at CanSecWest, HCSS, Devcon, Empire Hacking, GrrCon, LangSec, NorthSec, the O'Reilly Security Conference, PyCon, REcon, Security BSides, and SummerCon.

We specialize in software testing and code review assessments, supporting client organizations in the technology, defense, blockchain, and finance industries, as well as government entities. Notable clients include HashiCorp, Google, Microsoft, Western Digital, Uniswap, Solana, Ethereum Foundation, Linux Foundation, and Zoom.

To keep up with our latest news and announcements, please follow @trailofbits on X or LinkedIn and explore our public repositories at https://github.com/trailofbits. To engage us directly, visit our "Contact" page at https://www.trailofbits.com/contact or email us at info@trailofbits.com.

Trail of Bits, Inc.
228 Park Ave S #80688
New York, NY 10003
https://www.trailofbits.com
info@trailofbits.com

Notices and Remarks

Copyright and Distribution

Test Coverage Disclaimer

Trail of Bits performed all activities associated with this project in accordance with a statement of work and an agreed-upon project plan.

Security assessment projects are time-boxed and often rely on information provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

Trail of Bits uses automated testing techniques to rapidly test software controls and security properties. These techniques augment our manual security review work, but each has its limitations. For example, a tool may not generate a random edge case that violates a property or may not fully complete its analysis during the allotted time. A project's time and resource constraints also limit their use.